

Trabajo Práctico 3 - Bases de Datos^{*}

Alumno: Fabrizio Aguzzi

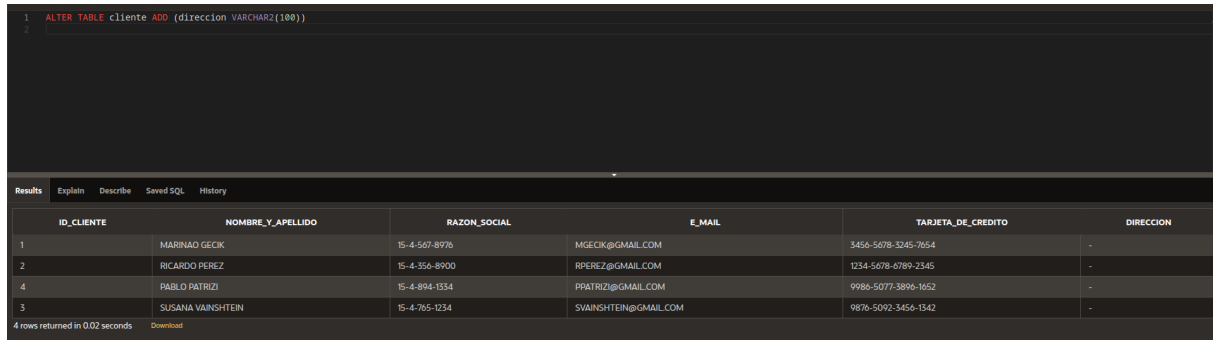
30 de marzo de 2025

- **Objetivos:** El objetivo del siguiente TP es poder crear y gestionar las sentencias de SQL Media que corresponden al ANSI SQL.
- **Consigna:** El alumno deberá subir un archivo en formato Word o PDF con las sentencias de cada uno de los 31 puntos que se detallan a continuación teniendo en cuenta el siguiente DER de la unidad 1, en caso de tener alguna duda o no poder hacer algún ejercicio comentarlo para revisión del docente.

^{*} Algunos ejercicios tienen Observaciones

1. Escribir una sentencia que agregue un campo a una tabla.

```
ALTER TABLE cliente ADD (direccion VARCHAR2(100))
```



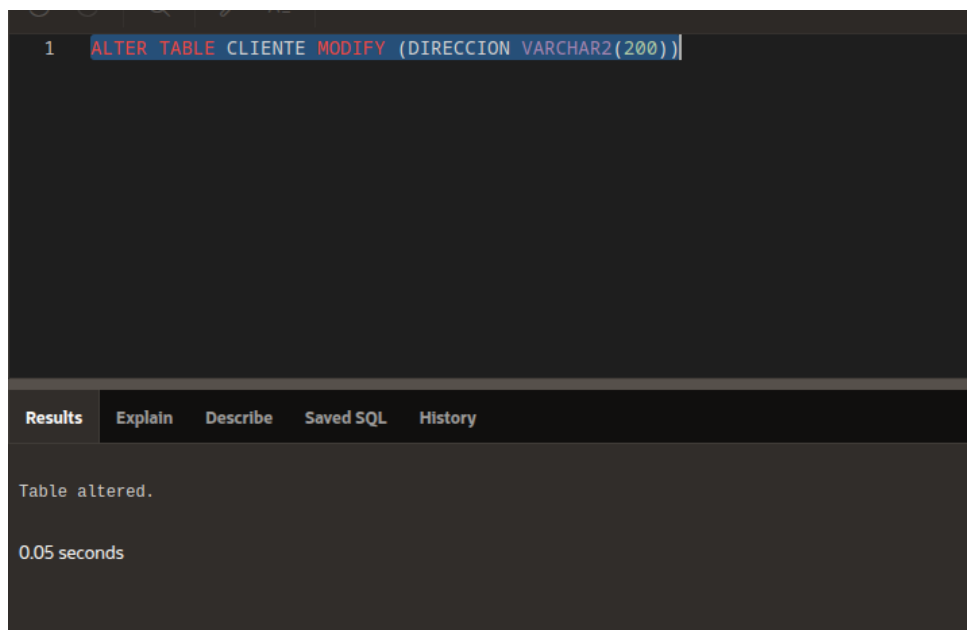
The screenshot shows a SQL Developer interface. The top pane contains the SQL statement: `1 ALTER TABLE cliente ADD (direccion VARCHAR2(100))`. The bottom pane shows the results of the query, which is a table with 6 columns: `ID_CLIENTE`, `NOMBRE_Y_APELLIDO`, `RAZON_SOCIAL`, `E_MAIL`, `TARJETA_DE_CREDITO`, and `DIRECCION`. The table contains 4 rows of data.

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO	DIRECCION
1	MARINAO GECK	15-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	-
2	RICARDO PEREZ	15-4-356-8900	RPerez@GMAIL.COM	1234-5678-6789-2345	-
4	PABLO PATRIZI	15-4-894-1334	PPATRIZI@GMAIL.COM	9986-5077-3896-1652	-
3	SUSANA VAINSTEIN	15-4-765-1234	SVAINSTEIN@GMAIL.COM	9876-5092-3456-1342	-

4 rows returned in 0.02 seconds

2. Escribir una sentencia que modifique un campo de una tabla.

```
ALTER TABLE CLIENTE MODIFY (DIRECCION VARCHAR2(200))
```



The screenshot shows a SQL Developer interface. The top pane contains the SQL statement: `1 ALTER TABLE CLIENTE MODIFY (DIRECCION VARCHAR2(200))`. The bottom pane shows the results of the query, which is a message: "Table altered." and the execution time: "0.05 seconds".

Results	Explain	Describe	Saved SQL	History
Table altered.				
0.05 seconds				

3. Escribir una sentencia que borre un campo de una tabla.

```
ALTER TABLE CLIENTE DROP COLUMN DIRECCION
```

```

1 ALTER TABLE CLIENTE DROP COLUMN DIRECCION
2
3 SELECT * FROM CLIENTE

```

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO
1	MARINAO GECIK	15-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654
2	RICARDO PEREZ	15-4-356-8900	RPEREZ@GMAIL.COM	1234-5678-6789-2345
4	PABLO PATRIZI	15-4-894-1334	PPATRIZI@GMAIL.COM	9986-5077-3896-1652
3	SUSANA VAINSHTEIN	15-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5092-3456-1342

4 rows returned in 0.01 seconds [Download](#)

4. Escribir una sentencia que haga que un campo se convierta en PRIMARY KEY.

OBSERVACIÓN : Una vez creada la tabla CLIENTE_RESTAURANT con su respectiva clave primaria, fue necesario eliminar la clave primaria primero antes de asignar otra.

```

CREATE TABLE CLIENTE_RESTAURANT (
  ID_CLIENTE VARCHAR2 (3) PRIMARY KEY ,
  Descripcion VARCHAR2 (255)
);

```

```

ALTER TABLE CLIENTE_RESTAURANT DROP PRIMARY KEY;

```

```

ALTER TABLE CLIENTE_RESTAURANT MODIFY DESCRIPCION PRIMARY KEY

```

```

DESC CLIENTE_RESTAURANT

```

```

1 CREATE TABLE CLIENTE_RESTAURANT (
2   ID_CLIENTE VARCHAR2 (3) PRIMARY KEY ,
3   Descripcion VARCHAR2 (255)
4 );
5
6 ALTER TABLE CLIENTE_RESTAURANT DROP PRIMARY KEY;
7
8
9 ALTER TABLE CLIENTE_RESTAURANT MODIFY DESCRIPCION PRIMARY KEY
10
11 DESC CLIENTE_RESTAURANT

```

Results	Explain	Describe	Saved SQL	History					
Object Type TABLE Object CLIENTE_RESTAURANT									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CLIENTE_RESTAURANT	ID_CLIENTE	VARCHAR2	3	-	-	-	✓	-	-
	DESCRIPCION	VARCHAR2	255	-	-	1	-	-	-

5. Escribir una sentencia que agregue una constraint a una tabla.

```

ALTER TABLE CLIENTE_RESTAURANT
ADD CONSTRAINT FK_CLIENTE_NEW FOREIGN KEY (ID_CLIENTE)
REFERENCES CLIENTE(ID_CLIENTE);

```

```
1 ALTER TABLE CLIENTE_RESTAURANT MODIFY (ID_CLIENTE NUMBER(22))
2
3 DESC CLIENTE
4
5 DESC CLIENTE_RESTAURANT
6
7 ALTER TABLE CLIENTE_RESTAURANT
8 ADD CONSTRAINT FK_CLIENTE_NEW FOREIGN KEY (ID_CLIENTE)
9 REFERENCES CLIENTE(ID_CLIENTE);
10
11
12
13
```

Results Explain Describe Saved SQL History

Table altered.

0.07 seconds

6. Escribir una sentencia que elimine una constraint que ya no se usa.

```
ALTER TABLE CLIENTE_RESTAURANT DROP CONSTRAINT FK_CLIENTE_NEW
```

```
11
12 ALTER TABLE CLIENTE_RESTAURANT DROP CONSTRAINT FK_CLIENTE_NEW
13
```

Results Explain Describe Saved SQL History

Table altered.

0.06 seconds

7. Escribir una sentencia que deshabilite una constraint pero que no la elimine.

```
ALTER TABLE CLIENTE_RESTAURANT DISABLE CONSTRAINT FK_CLIENTE_NEW
```

```
14 ALTER TABLE CLIENTE_RESTAURANT DISABLE CONSTRAINT FK_CLIENTE_NEW
```

Results Explain Describe Saved SQL History

Table altered.

0.05 seconds

8. Escribir una sentencia que habilite la constraint que estaba deshabilitada.

```
ALTER TABLE CLIENTE_RESTAURANT ENABLE CONSTRAINT FK_CLIENTE_NEW
```

```
13  
14 ALTER TABLE CLIENTE_RESTAURANT ENABLE CONSTRAINT FK_CLIENTE_NEW  
15  
16 DESC CLIENTE_RESTAURANT
```

Results Explain Describe Saved SQL History

Table altered.

0.05 seconds

9. Escribir una sentencia que elimine una tabla de la base de datos y que no quede en la “papelera de reciclaje”.

```
DROP TABLE CLIENTE_RESTAURANT PURGE;
```

```
25
26 DROP TABLE CLIENTE_RESTAURANT PURGE;
27
```

Results	Explain	Describe	Saved SQL	History
Table dropped.				
0.82 seconds				

10. ¿Para saber si la tabla está en la “papelera de reciclaje” de la base de datos cuál es la sentencia?

```
SELECT OBJECT_NAME, ORIGINAL_NAME, TYPE FROM USER_RECYCLEBIN
WHERE ORIGINAL_NAME = 'CLIENTE_RESTAURANT';
```

```
27
28 SELECT OBJECT_NAME, ORIGINAL_NAME, TYPE FROM USER_RECYCLEBIN
29 WHERE ORIGINAL_NAME = 'CLIENTE_RESTAURANT';
30
```

Results	Explain	Describe	Saved SQL	History
no data found				

11. Escriba la sentencia para traer el objeto que fue eliminado y está en la papelera.

OBSERVACIÓN : Aquí, al utilizar DROP TABLE me encontré con que la tabla eliminada NO se almacenó en la papelera.

```
FLASHBACK TABLE CLIENTE_RESTAURANT TO BEFORE DROP
```

```
6 DROP TABLE CLIENTE_RESTAURANT;  
7  
8  
9 FLASHBACK TABLE CLIENTE_RESTAURANT TO BEFORE DROP  
10  
11  
12 SELECT OBJECT_NAME, ORIGINAL_NAME, TYPE FROM USER_RECYCLEBIN WHERE ORIGINAL_NAME = 'CLIENTE_RESTAURANT';  
13
```

Results Explain Describe Saved SQL History

ORA-38305: object not in RECYCLE BIN

0.01 seconds

12. Escribir una sentencia que utilice el CREATE TABLE con un SELECT.

```
CREATE TABLE VIN02 AS SELECT * FROM VINO
```

```
15 CREATE TABLE VIN02 AS SELECT * FROM VINO
```

Results Explain Describe Saved SQL History

Table created.

0.10 seconds

13. Escribir una sentencia que combine el INSERT INTO con un SELECT.

```
INSERT INTO VIN02 SELECT * FROM VINO
```

```

14
15 INSERT INTO VINO2 SELECT * FROM VINO

```

Results Explain Describe Saved SQL History

9 row(s) inserted.

0.04 seconds

14. Escribir una sentencia que combine 2 SELECT, un SELECT por afuera y un SELECT en la parte del “FROM” para hacer un JOIN.

```

SELECT V1.DESCRIPCION, V1.ANIO_ELABORACION, V2.CATEGORIA, V2.BODEGA
FROM VINO V1
JOIN (
    SELECT CODIGO, CATEGORIA, BODEGA
    FROM VINO2
    WHERE CATEGORIA = 'TIN'
) V2 ON V1.CODIGO = V2.CODIGO;

```

```

19 SELECT V1.DESCRIPCION, V1.ANIO_ELABORACION, V2.CATEGORIA, V2.BODEGA
20 FROM VINO V1
21 JOIN (
22     SELECT CODIGO, CATEGORIA, BODEGA
23     FROM VINO2
24     WHERE CATEGORIA = 'TIN'
25 ) V2 ON V1.CODIGO = V2.CODIGO;
26

```

Results Explain Describe Saved SQL History

DESCRIPCION	ANIO_ELABORACION	CATEGORIA	BODEGA
MARTIN BRUNO MALBEC	1998	TIN	AF
LINDAFLOL MALBEC	2000	TIN	AL
VIA BLANCA MALBEC	1996	TIN	BA
MARTIN BRUNO MALBEC	1998	TIN	AF
LINDAFLOL MALBEC	2000	TIN	AL
VIA BLANCA MALBEC	1996	TIN	BA

6 rows returned in 0.02 seconds Download

15. Escribir una sentencia que combine un UPDATE con un SELECT. Y que actualice, además, no solo un campo sino más de uno.

OBSERVACIÓN : Utilicé ROWNUM = 1 porque la consulta estaba devolviendo más de una fila y eso daba error.


```

UPDATE VINO2
SET CATEGORIA = (SELECT CATEGORIA FROM VINO WHERE BODEGA = 'BA'
AND ROWNUM = 1),
ANIO_ELABORACION = (SELECT ANIO_ELABORACION FROM VINO WHERE
DESCRIPCION = 'MARTIN BRUNO MALBEC' AND ROWNUM = 1)
WHERE CODIGO = 'MM';

```

```

37
38 UPDATE VINO2
39 SET CATEGORIA = (SELECT CATEGORIA FROM VINO WHERE BODEGA = 'BA' AND ROWNUM = 1),
40 ANIO_ELABORACION = (SELECT ANIO_ELABORACION FROM VINO WHERE DESCRIPCION = 'MARTIN BRUNO MALBEC' AND ROWNUM = 1)
41 WHERE CODIGO = 'MM';
42
Results Explain Describe Saved SQL History
2 row(s) updated.
0.02 seconds

```

16. Escribir una sentencia DELETE combinada con SELECT.

```

DELETE FROM VINO2
WHERE ANIO_ELABORACION = (SELECT ANIO_ELABORACION
FROM VINO
WHERE CODIGO = 'MM')

```

```

43
44 DELETE FROM VINO2
45 WHERE ANIO_ELABORACION = (SELECT ANIO_ELABORACION
46 FROM VINO
47 WHERE CODIGO = 'MM')
48
Results Explain Describe Saved SQL History
4 row(s) deleted.
0.01 seconds

```

17. Escribir una sentencia que utilice el comando INSERT ALL.

```

INSERT ALL
  INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
  VALUES('LM', 'LINDAFLOR MALBEC', 2000, 'TIN', 'AL')
  INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
  VALUES('MM', 'MARTIN BRUNO MALBEC', 1998, 'TIN', 'AF')

```

```

        INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
        VALUES('VB', 'VIA BLANCA MALBEC', 1996, 'TIN', 'BA')
SELECT * FROM dual;

```

```

50 INSERT ALL
51 INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA) VALUES ('LM', 'LINDAFLOR MALBEC', 2000, 'TIN', 'AL')
52 INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA) VALUES ('MM', 'MARTIN BRUNO MALBEC', 1998, 'TIN', 'AF')
53 INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA) VALUES ('VB', 'VIA BLANCA MALBEC', 1996, 'TIN', 'BA')
54 SELECT * FROM dual;

```

Results Explain Describe Saved SQL History

3 row(s) inserted.

0.02 seconds

18. Escribir una sentencia que utilice el comando INSERT FIRST combinado con un SELECT.

```

INSERT FIRST
  WHEN ANIO_ELABORACION < 2000 THEN
    INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
    VALUES ('MM', 'MARTIN BRUNO MALBEC', 1998, 'TIN', 'AF')
  WHEN ANIO_ELABORACION >= 2000 THEN
    INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
    VALUES ('LM', 'LINDAFLOR MALBEC', 2000, 'TIN', 'AL')
SELECT ANIO_ELABORACION
FROM VINO;

```

```

50 INSERT FIRST
51 WHEN ANIO_ELABORACION < 2000 THEN
52 INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
53 VALUES ('MM', 'MARTIN BRUNO MALBEC', 1998, 'TIN', 'AF')
54 WHEN ANIO_ELABORACION >= 2000 THEN
55 INTO VINO2 (CODIGO, DESCRIPCION, ANIO_ELABORACION, CATEGORIA, BODEGA)
56 VALUES ('LM', 'LINDAFLOR MALBEC', 2000, 'TIN', 'AL')
57 SELECT ANIO_ELABORACION
58 FROM VINO;
59

```

Results Explain Describe Saved SQL History

9 row(s) inserted.

0.01 seconds

19. Escribir una sentencia que utilice le comando MERGE, que incluya INSERT y UP-DATE a la vez.

```

MERGE INTO cliente CLI
USING cliente_restaurant CLI2
ON (CLI.e_mail = CLI2.e_mail)
WHEN MATCHED THEN
    UPDATE SET CLI.razon_social = CLI2.razon_social
WHEN NOT MATCHED THEN
    INSERT (e_mail, nombre_y_apellido, razon_social, tarjeta_de_credito)
    VALUES (CLI2.e_mail, CLI2.nombre_y_apellido, CLI2.razon_social, CLI2.tarjeta_de_credito);

```

```

75
76 MERGE INTO cliente CLI
77 USING cliente_restaurant CLI2
78 ON (CLI.e_mail = CLI2.e_mail)
79 WHEN MATCHED THEN
80     UPDATE SET CLI.razon_social = CLI2.razon_social
81 WHEN NOT MATCHED THEN
82     INSERT (e_mail, nombre_y_apellido, razon_social, tarjeta_de_credito)
83     VALUES (CLI2.e_mail, CLI2.nombre_y_apellido, CLI2.razon_social, CLI2.tarjeta_de_credito);
84
Results Explain Describe Saved SQL History

4 row(s) updated.

0.13 seconds

```

20. Escribir una sentencia que utilice SELECT en el WHERE.

```

SELECT *
FROM cliente CLI,
(SELECT CLIE.tarjeta_de_credito
FROM cliente_restaurant CLIE
WHERE CLIE.razon_social = '15-4-894-1334')c2
WHERE CLI.TARJETA_DE_CREDITO = C2.TARJETA_DE_CREDITO

```

```

1 SELECT *
2 FROM cliente CLI,
3 (SELECT CLIE.tarjeta_de_credito
4 FROM cliente_restaurant CLIE
5 WHERE CLIE.razon_social = '15-4-894-1334')c2
6 WHERE CLI.TARJETA_DE_CREDITO = C2.TARJETA_DE_CREDITO
7
8
Results Explain Describe Saved SQL History

ID_CLIENTE NOMBRE_Y_APELLIDO RAZON_SOCIAL E_MAIL TARJETA_DE_CREDITO TARJETA_DE_CREDITO
4 PABLO PASTRIZ 15-4-894-1334 PASTRIZ@GMAIL.COM 9986-5077-5890-1032 9986-5077-5890-1032
1 rows returned in 0.05 seconds Download

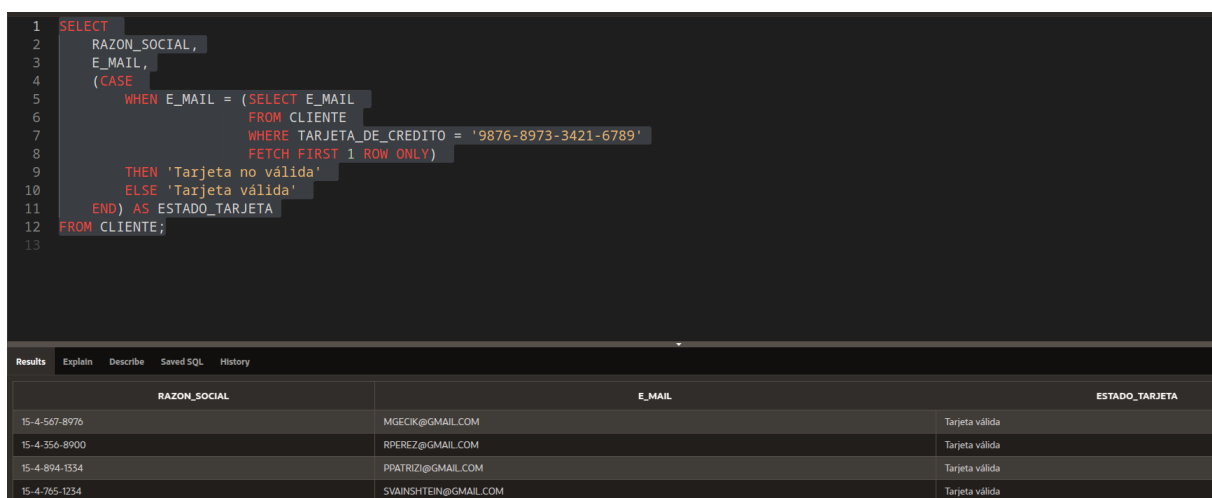
```

21. Escribir una sentencia de un SELECT dentro de un SELECT.

```

SELECT
    RAZON_SOCIAL,
    E_MAIL,
    (CASE
        WHEN E_MAIL = (SELECT E_MAIL
                        FROM CLIENTE
                        WHERE TARJETA_DE_CREDITO = '9876-8973-3421-6789'
                        FETCH FIRST 1 ROW ONLY)
        THEN 'Tarjeta no válida'
        ELSE 'Tarjeta válida'
    END) AS ESTADO_TARJETA
FROM CLIENTE;

```



```

1 SELECT
2   RAZON_SOCIAL,
3   E_MAIL,
4   (CASE
5       WHEN E_MAIL = (SELECT E_MAIL
6                       FROM CLIENTE
7                       WHERE TARJETA_DE_CREDITO = '9876-8973-3421-6789'
8                       FETCH FIRST 1 ROW ONLY)
9       THEN 'Tarjeta no válida'
10      ELSE 'Tarjeta válida'
11   END) AS ESTADO_TARJETA
12 FROM CLIENTE;
13

```

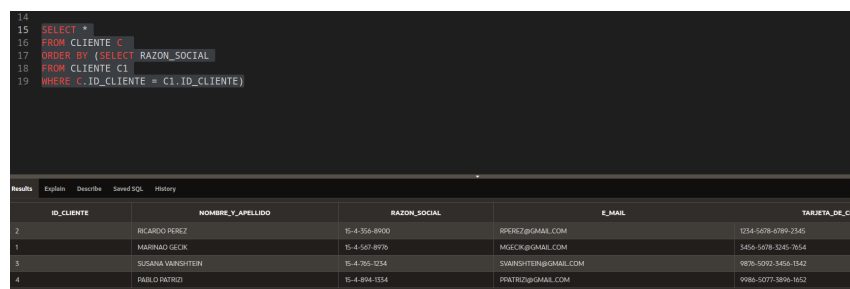
RAZON_SOCIAL	E_MAIL	ESTADO_TARJETA
15-4-567-8976	MGECKI@GMAIL.COM	Tarjeta válida
15-4-356-8900	RPEREZ@GMAIL.COM	Tarjeta válida
15-4-894-1334	PPATRIZ@GMAIL.COM	Tarjeta válida
15-4-765-1234	SVAINSHTEIN@GMAIL.COM	Tarjeta válida

22. Escribir una sentencia que utilice un SELECT en el ORDER BY.

```

SELECT *
FROM CLIENTE C
ORDER BY (SELECT RAZON_SOCIAL
FROM CLIENTE C1
WHERE C.ID_CLIENTE = C1.ID_CLIENTE)

```



```

14
15 SELECT *
16 FROM CLIENTE C
17 ORDER BY (SELECT RAZON_SOCIAL
18 FROM CLIENTE C1
19 WHERE C.ID_CLIENTE = C1.ID_CLIENTE)

```

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO
2	RICARDO PEREZ	15-4-356-8900	RPEREZ@GMAIL.COM	1234-5678-9101-2345
1	MARINAD GECK	15-4-567-8976	MGECKI@GMAIL.COM	3456-5678-3245-7654
3	SUSANA VAINSHTEIN	15-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5012-3456-1242
4	PABLO PATRIZ	15-4-894-1334	PPATRIZ@GMAIL.COM	0988-5077-3896-1652

23. Escribir una sentencia con una subconsulta que utilice un operador que no sea el “=”.

```

SELECT RAZON_SOCIAL, E_MAIL
FROM CLIENTE
WHERE TARJETA_DE_CREDITO IN (SELECT TARJETA_DE_CREDITO
                              FROM CLIENTE
                              WHERE E_MAIL LIKE '%GMAIL.COM');

```



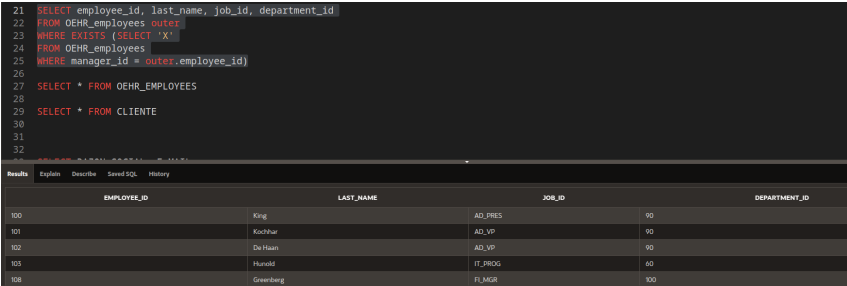
RAZON_SOCIAL	E_MAIL
15-4-567-8976	MGECK@gmail.com
15-4-556-8900	RPEREZ@gmail.com
15-4-894-1314	PRATRIZ@gmail.com
15-4-765-1234	SVANGHTEN@gmail.com

24. Escribir una sentencia que utilice los operadores EXISTS y NOT EXISTS.

```

SELECT employee_id, last_name, job_id, department_id
FROM OEHR_employees outer
WHERE EXISTS (SELECT 'X'
              FROM OEHR_employees
              WHERE manager_id = outer.employee_id)

```



EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90
103	Hunold	IT_PROG	80
108	Greenberg	PL_MGR	100

```

SELECT employee_id, last_name, job_id, department_id
FROM OEHR_employees outer
WHERE NOT EXISTS (SELECT 'X'
                  FROM OEHR_employees
                  WHERE manager_id = outer.employee_id)

```

```

21 SELECT employee_id, last_name, job_id, department_id
22 FROM OEHR_employees outer
23 WHERE NOT EXISTS (SELECT 'X'
24 FROM OEHR_employees
25 WHERE manager_id = outer.employee_id)
26
27 SELECT * FROM OEHR_EMPLOYEES
28
29 SELECT * FROM CLIENTE
30
31
32

```

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
104	Ernst	IT_PROG	60
105	Austin	IT_PROG	60
106	Pataballa	IT_PROG	60
107	Lorentz	IT_PROG	60
109	Favret	FI_ACCOUNT	100

25. Escribir una sentencia que utilice un DELETE con un SELECT.

```

DELETE FROM CLIENTE_RESTAURANT
WHERE RAZON_SOCIAL = (SELECT RAZON_SOCIAL FROM CLIENTE_RESTAURANT
WHERE TARJETA_DE_CREDITO = '9986-5077-3896-1652')

```

```

39
40 DELETE FROM CLIENTE_RESTAURANT
41 WHERE RAZON_SOCIAL = (SELECT RAZON_SOCIAL FROM CLIENTE_RESTAURANT
42 WHERE TARJETA_DE_CREDITO = '9986-5077-3896-1652')

```

Results	Explain	Describe	Saved SQL	History
1 row(s) deleted.				
0.01 seconds				

26. Escribir una sentencia que utilice la cláusula WITH de Oracle.

```

WITH
DEPT_COSTS AS (
SELECT D.department_name, SUM(E.salary)
as dept_total
FROM OEHR_employees E, OEHR_departments D
WHERE E.department_id = D.department_id
GROUP BY department_name),
AVG_COST AS (
SELECT SUM (dept_total) /COUNT (*) AS
dept_avg
FROM DEPT_COSTS)
SELECT *
FROM dept_costs
WHERE dept_total > (SELECT dept_avg FROM avg_cost)
ORDER BY department_name

```

```

47 WITH
48 DEPT_COSTS AS (
49   SELECT D.department_name, SUM(E.salary)
50   AS dept_total
51   FROM OEHR_employees E, OEHR_departments D
52   WHERE E.department_id = D.department_id
53   GROUP BY department_name),
54 AVG_COST AS (
55   SELECT SUM (dept_total) /COUNT (*) AS
56   dept_avg
57   FROM DEPT_COSTS)
58 SELECT *
59 FROM dept_costs
60 WHERE dept_total > (SELECT dept_avg FROM avg_cost)
61 ORDER BY department_name

```

DEPARTMENT_NAME	DEPT_TOTAL
Sales	304500
Shipping	156400

2 rows returned in 0.02 seconds [Download](#)

27. Escribir una sentencia que utilice el CONNECT BY PRIOR, esto no va a tener sentido sino utilizas una estructura de tabla como la de “oehr_employees”.

```

SELECT last_name||' reports to '||
PRIOR last_name "WALK TOP DOWN"
FROM OEHR_employees
START WITH last_NAME = 'King'
CONNECT BY PRIOR employee_id = manager_id

```

```

64
65 SELECT last_name||' reports to '||
66 PRIOR last_name "WALK TOP DOWN"
67 FROM OEHR_employees
68 START WITH last_NAME = 'King'
69 CONNECT BY PRIOR employee_id = manager_id
70

```

WALK TOP DOWN
King reports to
King reports to
Kochhar reports to King
Greenberg reports to Kochhar
Faviet reports to Greenberg

Copyright © 1999, 2024, Oracle and/or its affiliates.

28. Escribir una sentencia que utilice el operador set UNION ALL.

```

SELECT RAZON_SOCIAL,E_MAIL FROM cliente
UNION ALL
SELECT RAZON_SOCIAL,E_MAIL FROM CLIENTE_RESTAURANT

```

```

72
73
74 SELECT RAZON_SOCIAL,E_MAIL FROM cliente
75 UNION ALL
76 SELECT RAZON_SOCIAL,E_MAIL FROM CLIENTE_RESTAURANT

```

RAZON_SOCIAL	E_MAIL
15-4-567-8976	MGECK@GMAIL.COM
15-4-356-8900	RPEREZ@GMAIL.COM
15-4-894-1334	PPATRIZ@GMAIL.COM
15-4-765-1234	SVAINSHTEIN@GMAIL.COM
15-4-567-8976	MGECK@GMAIL.COM

29. Escribir una sentencia que utilice el operador set UNION.

```

SELECT RAZON_SOCIAL,E_MAIL FROM cliente
UNION
SELECT RAZON_SOCIAL,E_MAIL FROM CLIENTE_RESTAURANT

```

```

73
74 SELECT RAZON_SOCIAL,E_MAIL FROM cliente
75 UNION
76 SELECT RAZON_SOCIAL,E_MAIL FROM CLIENTE_RESTAURANT

```

RAZON_SOCIAL	E_MAIL
15-4-356-8900	RPEREZ@GMAIL.COM
15-4-567-8976	MGECK@GMAIL.COM
15-4-765-1234	SVAINSHTEIN@GMAIL.COM
15-4-894-1334	PPATRIZ@GMAIL.COM

4 rows returned in 0.01 seconds [Download](#)

30. Escribir una sentencia que utilice el operador set INTERSECT.

Escribir una sentencia que utilice el operador set INTERSECT.

```

73
74 SELECT RAZON_SOCIAL,E_MAIL FROM cliente
75 INTERSECT
76 SELECT RAZON_SOCIAL,E_MAIL FROM CLIENTE_RESTAURANT

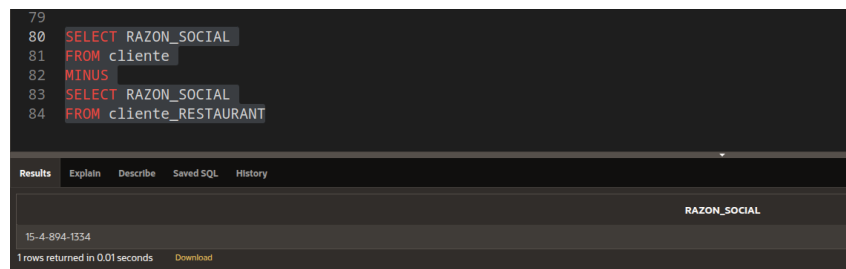
```

RAZON_SOCIAL	E_MAIL
15-4-356-8900	RPEREZ@GMAIL.COM
15-4-567-8976	MGECK@GMAIL.COM
15-4-765-1234	SVAINSHTEIN@GMAIL.COM

3 rows returned in 0.00 seconds [Download](#)

31. Escribir una sentencia que utilice el operador set MINUS.

```
SELECT RAZON_SOCIAL
FROM cliente
MINUS
SELECT RAZON_SOCIAL
FROM cliente_RESTAURANT
```



The screenshot shows a SQL query execution interface. The query is as follows:

```
79
80 SELECT RAZON_SOCIAL
81 FROM cliente
82 MINUS
83 SELECT RAZON_SOCIAL
84 FROM cliente_RESTAURANT
```

Below the query, there are tabs for 'Results', 'Explain', 'Describe', 'Saved SQL', and 'History'. The 'Results' tab is selected, showing a table with one column, 'RAZON_SOCIAL'. The table contains one row with the value '15-4-894-1334'. At the bottom, it states '1 rows returned in 0.01 seconds' and provides a 'Download' link.

RAZON_SOCIAL
15-4-894-1334